

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования

**«Пермский национальный исследовательский политехнический
университет»**

Электротехнический факультет

Выпускающая кафедра: Информационные технологии и
автоматизированные системы (ИТАС)

Направление подготовки: 09.03.04 Программная инженерия

ОТЧЕТ

Лабораторная работа №8

«Программа, управляемая событиями.»

По дисциплине «Основы алгоритмизации и программирования»

Вариант 15

Выполнил: студент группы РИС-25-2Б

Ильинов Фёдор Михайлович

Принял: Доц. Полякова О. А.

Пермь 2026

ПОСТАНОВКА ЗАДАЧИ

1. Определить иерархию пользовательских классов (см. лабораторную работу №5). Во главе иерархии должен стоять абстрактный класс с чисто виртуальными методами для ввода и вывода информации об атрибутах объектов.

2. Реализовать конструкторы, деструктор, операцию присваивания, селекторы и модификаторы.

3. Определить класс-группу на основе структуры, указанной в варианте.

4. Для группы реализовать конструкторы, деструктор, методы для добавления и удаления элементов в группу, метод для просмотра группы, перегрузить операцию для получения информации о размере группы.

5. Определить класс Диалог – наследника группы, в котором реализовать методы для обработки событий.

6. Добавить методы для обработки событий группой и объектами пользовательских классов.

7. Написать тестирующую программу.

8. Нарисовать диаграмму классов и диаграмму объектов.

Базовый класс:

ПЕЧАТНОЕ_ИЗДАНИЕ(PRINT)

Название– string

Автор – string

Производный класс

ЖУРНАЛ (MAGAZIN)

Количество страниц — int

Группа – Дерево (Tree).

Команды:

1. Создать группу (формат команды: m количество элементов группы).
2. Добавить элемент в группу (формат команды: +)
3. Удалить элемент из группы (формат команды -)
4. Вывести информацию об элементах группы (формат команды: s)
5. Вывести информацию о названиях всех элементов группы (формат команды : z)
6. Конец работы (формат команды: q)

АНАЛИЗ.

1. Внутри main() создаётся объект класса Dialog, после создания которого вызывается метод Dialog::Execute(), который переводит программу в интерактивный режим обработки пользовательских команд.

2. Метод реализует основной цикл работы программы. Цикл продолжается до тех пор, пока не будет установлен флаг завершения (EndState).

3. Внутри цикла вызывается GetEvent(), который осуществляет чтение пользовательского ввода и формирует структуру Event.

4. После формирования события оно передаётся в handleEvent(), где происходит его обработка:

5. Класс Vector реализует динамический массив указателей Object*.

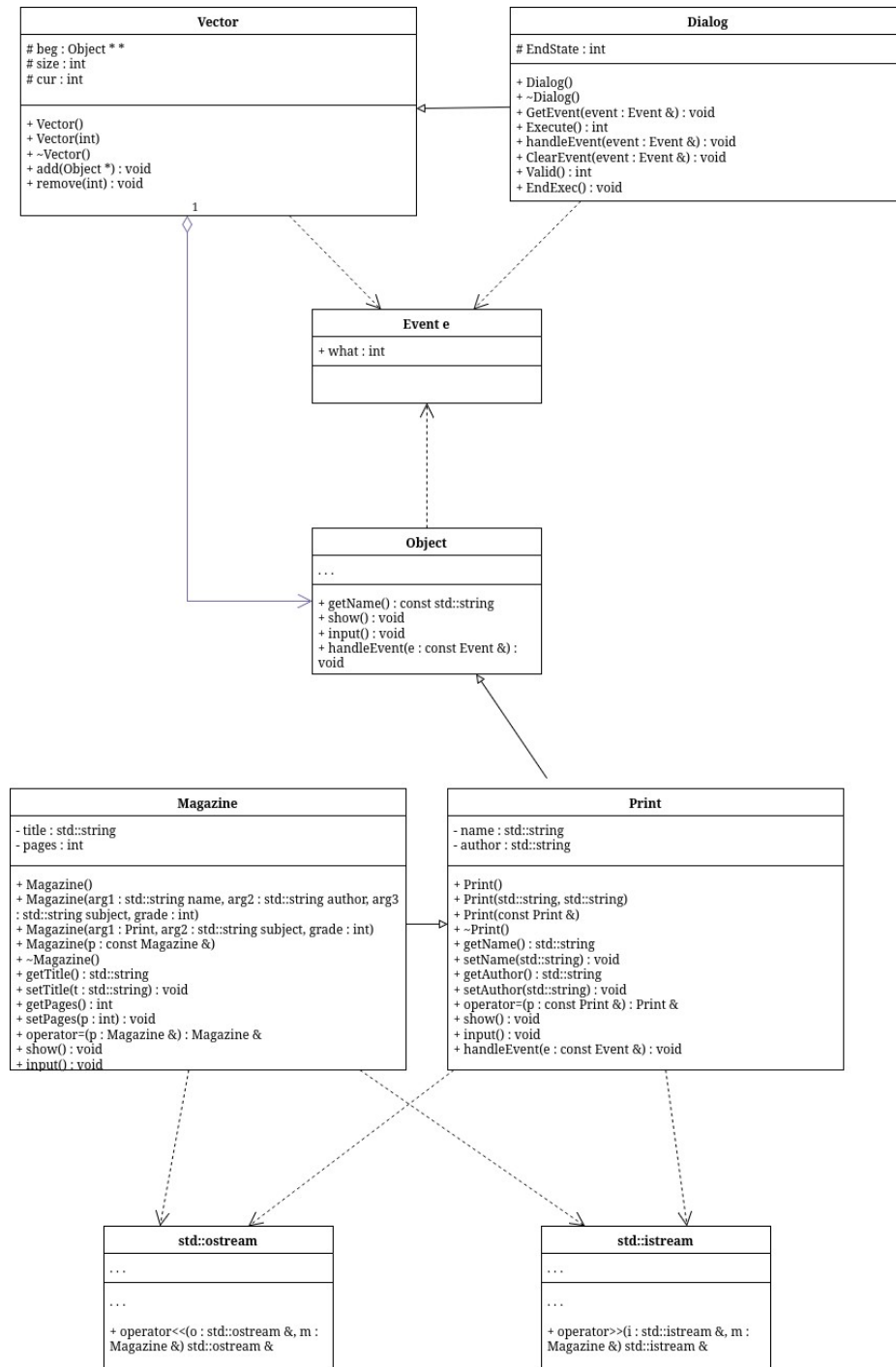
6. Object является абстрактным базовым классом и определяет общий интерфейс.

7. Класс Print наследует Object и содержит поля name и author и реализует ввод/вывод данных через перегруженные операторы, а также метод show() для отображения информации.

8. Класс Magazine наследует Print и расширяет его.

9. Event представляет собой структуру обмена данными между Dialog, Vector и объектами.. Он содержит набор констант команд, соответствующих действиям пользователя и используется как единый механизм передачи управляющих сигналов по системе.

UML-ДИАГРАММА



КОД Dialog.h

```
#ifndef ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_DIALOG_H
#define ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_DIALOG_H
#include "Event.h"
#include "Vector.h"

class Dialog : public Vector {
public:
    Dialog();
    virtual ~Dialog();
    virtual void GetEvent(Event &event);
    virtual int Execute();
    virtual void handleEvent(Event &event);
    virtual void ClearEvent(Event &event);
    int Valid();
    void EndExec();
protected:
    int EndState;
};

#endif //ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_DIALOG_H
```

КОД Dialog.cpp

```
#include "Dialog.h"
#include <iostream>
#include "Magazine.h"

Dialog::Dialog() : Vector() {
    EndState = 0;
}

Dialog::~Dialog() {

}

void Dialog::GetEvent(Event &event) {
    std::string OpInt = "+-s/qam";
    std::string s;
    std::string param;
    char code;
    std::cout << "> ";
```

```

std::cin >> s;
code = s[0];
if (OpInt.find(code) >= 0)
{
    std::string param;
    event.what = evMessage;
    switch (code) {
        case 'm': {
            std::cout << "Size? ";
            std::cin >> param;
            event.a = atoi(param.c_str());
            event.command = cmMake;
            break;
        }
        case '+': {
            std::cout << "Type? Magazine (m) Print (p) ";
            std::cin >> param;
            if (param[0] != typeMagazine && param[0] != typePrint) {
                std::cout << "Unknown type\n";
                event.what = evNothing;
                break;
            }
            event.command = cmAdd;
            event.a = param[0];
            break;
        }
        case '-': {
            std::cout << "Number? ";
            std::cin >> param;
            event.a = atoi(param.c_str());
            event.a--;
            event.command = cmDel;
            break;
        }
        case 's':
            event.command = cmShow;
            break;
        case 'q':
            event.command = cmQuit;
            break;
        case 'z':
            event.command = cmShowNames;
            break;
    }
} else {

```

```

        event.what = evNothing;
        std::cout << "Unknown command\n";
    }
}

```

```

int Dialog::Execute() {
    Event event;
    do {
        EndState = 0;
        GetEvent(event);
        handleEvent(event);
    } while (!Valid());
    return EndState;
}

```

```

int Dialog::Valid() {
    if (EndState == 0) return 0;
    return 1;
}

```

```

void Dialog::ClearEvent(Event &event) {
    event.what = evNothing;
}

```

```

void Dialog::EndExec() {
    EndState = 1;
}

```

```

void Dialog::handleEvent(Event &event) {
    if (event.what == evMessage) {
        switch (event.command) {
            case cmMake:
                size = event.a;
                beg = new Object *[size];
                cur = 0;
                ClearEvent(event);
                break;
            case cmAdd:
            {
                switch (event.a) {
                    case typePrint: {
                        Print *p = new Print;
                        std::cin >> *p;
                        add(p);
                        break;

```

```

    }
    case typeMagazine:{
        Magazine *p = new Magazine;
        std::cin >> *p;
        add(p);
        break;
    }
}
ClearEvent(event);
break;
}
case cmDel:
    remove(event.a);
    ClearEvent(event);
    break;
case cmShow:
    std::cout << *this;
    ClearEvent(event);
    break;
case cmQuit: EndExec();
    ClearEvent(event);
    break;
case cmShowNames: {
    for (int i = 0; i < cur; i++) {
        std::cout << i + 1 << ' ' << beg[i]->getName() << '\n';
    }
}
default: Vector::handleEvent(event);
};
}

```

КОД Event.h

```
#ifndef ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_EVENT_H
#define ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_EVENT_H

const int evNothing = 0;
const int evMessage = 100;
const int cmAdd = 1;
const int cmDel = 2;
const int cmShow = 4;
const int cmShowNames = 5;
const int cmMake = 6;
const int cmQuit = -1;

const int typeMagazine = 'm';
const int typePrint = 'p';

struct Event {
    int what;
    union {
        int command;
        struct {
            int message;
            int a;
        };
    };
};

#endif //ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_EVENT_H
```

КОД Magazine.h

```
#ifndef ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_Student_H
#define ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_Student_H
#include <string>
#include "Print.h"

class Magazine : public Print {
    std::string title;
    int pages;

public:
    Magazine();
    Magazine(std::string name, std::string author, std::string subject, int grade);
    Magazine(Print, std::string subject, int grade);
    Magazine(Magazine&);
    ~Magazine();
    std::string getTitle();
    void setTitle(std::string);
    int getPages();
    void setPages(int);
    Magazine &operator=(Magazine &p);
    friend std::ostream &operator<<(std::ostream &, Magazine &);
    friend std::istream &operator>>(std::istream &, Magazine &);
    void show() override;
    void input() override;
};

#endif //ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_Student_H
```

КОД Magazine.cpp

```
#ifndef ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_Student_H
#define ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_Student_H
#include <string>
#include "Print.h"

class Magazine : public Print {
    std::string title;
    int pages;

public:
    Magazine();
    Magazine(std::string name, std::string author, std::string subject, int grade);
    Magazine(Print, std::string subject, int grade);
    Magazine(Magazine&);
    ~Magazine();
    std::string getTitle();
    void setTitle(std::string);
    int getPages();
    void setPages(int);
    Magazine &operator=(Magazine &p);
    friend std::ostream &operator<<(std::ostream &, Magazine &);
    friend std::istream &operator>>(std::istream &, Magazine &);
    void show() override;
    void input() override;
};

#endif //ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_Student_H
```

КОД Object.h

```
#ifndef ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_OBJECT_H
#define ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_OBJECT_H
#include "Event.h"

class Object {
public:
    virtual void show() = 0;
    virtual void input() = 0;
    virtual void handleEvent(const Event &e)=0;
    virtual const std::string getName() = 0;
};

#endif //ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_OBJECT_H
```

КОД Print.h

```
#ifndef ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_PERSON_H
#define ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_PERSON_H
#include <string>
#include "Object.h"

class Print : public Object {
    std::string name;
    std::string author;

public:
    Print();
    Print(std::string, std::string);
    Print(Print &);
    ~Print();
    const std::string getName() override;
    void setName(std::string);
    const std::string getAuthor();
    void setAuthor(std::string);
    Print &operator=(Print const &p);
    friend std::ostream &operator<<(std::ostream &, const Print &);
    friend std::istream &operator>>(std::istream &, Print &);
    void show() override;
    void input() override;

    virtual void handleEvent(const Event &e);
};

#endif //ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_PERSON_H
```

КОД Print.cpp

```
#include "Print.h"
#include <iostream>

Print::Print() {
    name = "Unknown";
    author = 18;
}

Print::Print(std::string s, std::string a) {
    name = s;
    author = a;
}

Print::Print(Print & p) {
    name = p.name;
    author = p.author;
}

Print::~~Print() {
    name = "Unknown";
    author = 18;
}

const std::string Print::getName() {
    return name;
}

void Print::setName(std::string s) {
    name = s;
}

const std::string Print::getAuthor() {
    return author;
}

void Print::setAuthor(std::string a) {
    author = a;
}

Print &Print::operator=(Print const &p) {
    name = p.name;
```

```

    author = p.author;
    return *this;
}

void Print::show() {
    std::cout << *this;
}

void Print::input() {
    std::cin >> *this;
}

void Print::handleEvent(const Event &e) {
}

std::ostream &operator<<(std::ostream &stream, const Print &p) {
    std::cout << "Print: ";
    std::cout << "\tName: " << p.name;
    std::cout << "\tAuthor: " << p.author;
    return stream;
}

std::istream &operator>>(std::istream &stream, Print &p) {
    std::cout << "Name? ";
    stream >> p.name;
    std::cout << "Author? ";
    stream >> p.author;
    return stream;
}

```

КОД Vector.h

```
#include "Print.h"
#include <iostream>

Print::Print() {
    name = "Unknown";
    author = 18;
}

Print::Print(std::string s, std::string a) {
    name = s;
    author = a;
}

Print::Print(Print & p) {
    name = p.name;
    author = p.author;
}

Print::~~Print() {
    name = "Unknown";
    author = 18;
}

const std::string Print::getName() {
    return name;
}

void Print::setName(std::string s) {
    name = s;
}

const std::string Print::getAuthor() {
    return author;
}

void Print::setAuthor(std::string a) {
    author = a;
}

Print &Print::operator=(Print const &p) {
    name = p.name;
```

```

    author = p.author;
    return *this;
}

void Print::show() {
    std::cout << *this;
}

void Print::input() {
    std::cin >> *this;
}

void Print::handleEvent(const Event &e) {
}

std::ostream &operator<<(std::ostream &stream, const Print &p) {
    std::cout << "Print: ";
    std::cout << "\tName: " << p.name;
    std::cout << "\tAuthor: " << p.author;
    return stream;
}

std::istream &operator>>(std::istream &stream, Print &p) {
    std::cout << "Name? ";
    stream >> p.name;
    std::cout << "Author? ";
    stream >> p.author;
    return stream;
}

```

КОД Vector.cpp

```
#include "Vector.h"

#include <iostream>

Vector::Vector() {
    beg = 0;
    size = 0;
    cur = 0;
}

Vector::~Vector() {
    if (beg != 0) delete [] beg;
    beg = 0;
}

Vector::Vector(int n) {
    beg = new Object *[n];
    cur = 0;
    size = n;
}

void Vector::add(Object *p) {
    if (cur >= size) {
        int newSize = (size == 0) ? 1 : size * 2;

        Object **newArr = new Object *[newSize];

        for (int i = 0; i < cur; ++i)
            newArr[i] = beg[i];

        delete[] beg;

        beg = newArr;
        size = newSize;
    }

    beg[cur++] = p;
}

void Vector::remove(int idx) {
    for (int i = idx + 1; i < cur; i++)
```

```

        beg[i - 1] = beg[i];
        beg[cur--] = nullptr;
    }

std::ostream &operator<<(std::ostream &out, const Vector &v) {
    if (v.size == 0)
        std::cout << "Empty\n";
    Object **p = v.beg;
    for (int i = 0; i < v.cur; i++) {
        p[i]->show();
        std::cout << '\n';
    }

    return out;
}

void Vector::handleEvent(const Event &e) {
    if (e.what == evMessage) {
        Object **p = beg;
        for (int i = 0; i < cur; i++) {
            (*p)->handleEvent(e);
            p++;
        }
    }
}

```

КОД main.cpp

```
#include <iostream>

#include "Dialog.h"

using namespace std;

int main() {
    Dialog d;
    d.Execute();
}
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ ПРОГРАММЫ

```
> m
Size? 10
> +
Type? Magazine (m) Print (p) m
Name? Modern_Physics_Review
Author? Albert_Newton
Title? Quantum_Insights_April_Edition
Pages? 98
> +
Type? Magazine (m) Print (p) m
Name? Daily_Tech_Chronicle
Author? Emily_Stone
Title? AI_And_Future_Trends_Special
Pages? 2
> +
Type? Magazine (m) Print (p) m
Name? Classic_Literature_Archive
```

Author? William_Blake

Title? Victorian_Era_Studies_Issue_12

Pages? 210

> +

Type? Magazine (m) Print (p) p

Name? Computer_Science_Notes

Author? Alan_Turing

> +

Type? Magazine (m) Print (p) p

Name? Space_Exploration_Digest

Author? Carl_Sagan

> s

Magazine:

Print's name: Modern_Physics_Review

Print's author: Albert_Newton

Magazine's title: Quantum_Insights_April_Edition

Number of pages: 98

Magazine:

Print's name: Daily_Tech_Chronicle

Print's author: Emily_Stone

Magazine's title: AI_And_Future_Trends_Special

Number of pages: 2

Magazine:

Print's name: Classic_Literature_Archive

Print's author: William_Blake

Magazine's title: Victorian_Era_Studies_Issue_12

Number of pages: 210

Print:

Name: Computer_Science_Notes

Author: Alan_Turing

Print:

Name: Space_Exploration_Digest

Author: Carl_Sagan

> z

1 Modern_Physics_Review

2 Daily_Tech_Chronicle

3 Classic_Literature_Archive

4 Computer_Science_Notes

5 Space_Exploration_Digest

> -

Number? 2

> -

Number? 2

> s

Magazine:

Print's name: Modern_Physics_Review

Print's author: Albert_Newton

Magazine's title: Quantum_Insights_April_Edition

Number of pages: 98

Print:

Name: Computer_Science_Notes

Author: Alan_Turing

Print:

Name: Space_Exploration_Digest

Author: Carl_Sagan

> z

1 Modern_Physics_Review

2 Computer_Science_Notes

3 Space_Exploration_Digest

> q

Process finished with exit code 0

ВОПРОСЫ

1. Что такое класс-группа? Привести примеры таких классов.

Класс-группа — это класс, предназначенный для хранения и управления набором объектов базового класса. Примеры: Vector (массив объектов), List (список), Tree (дерево).

2. Привести пример описания класса-группы Список (List).

Класс List представляет динамическую структуру данных, содержащую указатели на объекты базового класса Object и обеспечивающую операции добавления, удаления и обхода элементов.

3. Привести пример конструктора (с параметром, без параметров, копирования) для класса-группы Список.

Конструктор без параметров создаёт пустой список, с параметром задаёт начальный размер, конструктор копирования создаёт новый список как копию существующего с выделением новой памяти.

4. Привести пример деструктора для класса-группы Список.

Деструктор освобождает динамически выделенную память всех элементов списка и самого массива указателей.

5. Привести пример метода для просмотра элементов для класса-группы Список.

Метод просмотра проходит по всем элементам списка и вызывает у каждого объекта метод Show().

6. Какой вид иерархии дает группа?

Группа даёт композиционную иерархию объектов с общим базовым классом.

7. Почему во главе иерархии классов, содержащихся в группе объектов должен находиться абстрактный класс?

Потому что абстрактный класс задаёт общий интерфейс для всех производных классов и обеспечивает полиморфизм.

8. Что такое событие? Для чего используются события?

Событие — это структура данных, описывающая действие или команду. Используется для передачи управляющих сигналов между компонентами программы.

9. Какие характеристики должно иметь событие-сообщение?

Оно должно содержать тип события, код команды и при необходимости дополнительные параметры.

10. Привести пример структуры, описывающей событие.

Структура TEvent с полями what и union, содержащим command или параметры команды.

11. Задана структура события `struct TEvent { int what; union { MouseEvent mouse; KeyDownEvent keyDown; MessageEvent message; } }`; Какие значения, и в каких случаях присваиваются полю what?

Поле what определяет тип события и принимает значения в зависимости от источника: мышь, клавиатура или сообщение.

12. Задана структура события `struct TEvent { int what; union { int command; struct { int message; int a; }; } }`; Какие значения, и в каких случаях присваиваются полю command?

Поле `command` заполняется при событии-команде и содержит код выполняемой операции.

13. Для чего используются поля `a` и `message`?

Поле `message` содержит тип сообщения, поле `a` — дополнительный параметр команды.

14. Какие методы необходимы для организации обработки сообщений?

Методы `GetEvent()`, `HandleEvent()`, `ClearEvent()`, `Valid()`, `EndExec()`.

15. Какой вид имеет главный цикл обработки событий-сообщений?

Цикл получает событие, обрабатывает его и повторяется до установки признака завершения.

16. Какую функцию выполняет метод `ClearEvent()`? Каким образом?

Очищает текущее событие, сбрасывая его поля в состояние отсутствия события.

17. Какую функцию выполняет метод `HandleEvent()`? Каким образом?

Обрабатывает событие, передавая его соответствующим объектам или выполняя команду.

18. Какую функцию выполняет метод `GetEvent()`?

Считывает ввод пользователя и формирует структуру события `TEvent`.

19. Для чего используется поле EndState? Какой класс (объект) содержит это поле?

EndState используется для завершения работы программы, содержится в классе Dialog.

20. Для чего используется функция Valid()?

Для проверки корректности текущего состояния события или объекта перед обработкой.